INDIA RUNS · TRACK 1 — THE DATA & AI CHALLENGE

Intelligent Candidate Discovery

Beyond keyword filters: an explainable, signal-driven candidate ranking engine.

TEAM NAME	<your team name>
PROBLEM STATEMENT	Data & AI Challenge — Intelligent Candidate Discovery
TEAM LEADER NAME	<your name>

Solution Overview

What is the proposed solution?

An explainable, modular candidate-ranking pipeline that scores every candidate against a job posting across four signal types — contextual semantic relevance, skill coverage, experience fit, and behavioral / intent signals — and returns a ranked shortlist with a plain-English justification for every score.

What differentiates this approach from traditional matching systems?

Beyond keywords

Semantic match vs the JD AND a synthesized 'ideal candidate' profile — surfaces 'hidden gems' phrased differently from the posting.

Behavioral / intent signals

Models who is reachable right now via activity recency, upskilling, and engagement — most systems skip this entirely.

Fully explainable

Every score = 4 components + matched/missing skills + a plain-English justification. No black box.

Hallucination-safe

Rule-based justifications are template-filled from facts; the optional LLM layer is grounding-checked with automatic fallback.

Config-driven

Zero code changes to point at a new dataset, or to swap the embedding layer for a production model.

JD Understanding & Candidate Evaluation

Key requirements extracted from every JD

- **Required vs. preferred skills**

  via a 25-skill taxonomy with synonyms/abbreviations (e.g. "ML" ↔ "machine learning"), classified using sentence-level cue detection.

- **Experience range**

  min/max years extracted via pattern matching (e.g. "2-5 years", "3+ years").

- **Seniority level**

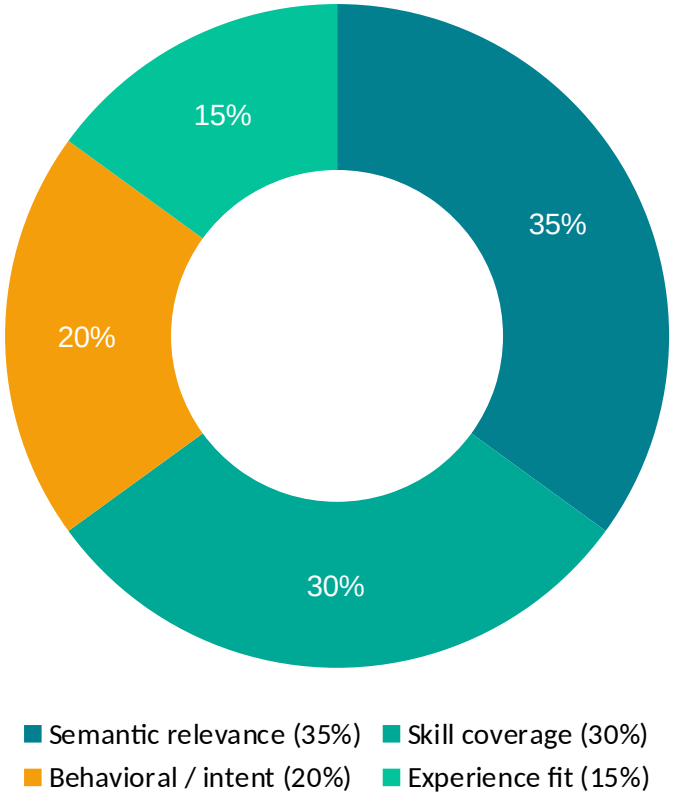
  junior / mid / senior / lead, via keyword detection.

- **Hypothetical ideal candidate profile**

  a synthesized skills-forward description used as a SECOND comparison target for semantic matching.

How candidate fit is evaluated beyond keyword matching

Four complementary signals, combined into final_score. Behavioral/intent (20%) is the signal most systems skip — it captures who is reachable right now.



Ranking Methodology

```
final_score = 0.35 × semantic + 0.30 × skill_match + 0.15 × experience_fit + 0.20 × behavioral
```

(× 0.85 confidence discount if the profile is flagged sparse_profile — see slide 5)

How candidates are retrieved, scored & ranked

Every candidate in the pool is scored against every job — a complete re-score, not a coarse pre-filter. At this scale (~45 candidates × 3 jobs) this runs in under 2 seconds, and guarantees no 'hidden gem' is filtered out before scoring.

Models, algorithms & heuristics used

- TF-IDF + Truncated SVD (LSA) for semantic embeddings — offline, no model downloads
- Rule-based sentence-cue NLP for required vs. preferred skill classification
- Taxonomy-based skill extraction with word-boundary matching
- Trapezoidal experience-fit curve (steep penalty if under-qualified, gentle if over)
- Pool-relative min-max normalization for behavioral signals
- Optional: grounded Claude re-ranking for top-N, with hallucination rejection

Explainability & Data Validation

How decisions are explained

Every justification is built ENTIRELY from pre-computed fields: contextual-fit tier, matched/missing required skills, experience-vs-range, and behavioral highlights. A template fill, not free generation — zero hallucination risk by construction.

Preventing hallucinations

The optional LLM step's output is re-scanned with the same taxonomy skill extractor used everywhere else. If it asserts a skill outside the 'grounding set' (JD skills + candidate's own detected skills), that justification is REJECTED and the rule-based one is used instead.

Human stays in the loop

Every output row carries a justification_source (rule_based / llm) flag. Quality flags below are SURFACED for recruiter review, not silently acted on — except the sparse-profile score discount, where confidence is genuinely low.

Handling inconsistent, low-quality, or suspicious profiles

Flag	Trigger	Effect
sparse_profile	Resume text under 8 words — too little to assess	Final score discounted ×0.85, noted in justification
implausible_experience_value	years_experience negative or > 45	Surfaced for review (no auto-penalty)
skills_not_substantiated_in_narrative	>80% of listed skills never appear in the resume narrative	Surfaced for review (possible keyword-stuffing)
possible_duplicate_profile	Resume text near-identical to another candidate's	Surfaced for review, noted in justification (possible spam/fraud)

End-to-End Workflow

From job description input to a ranked, explainable shortlist

1

Input

Job descriptions, candidate profiles, behavioral signals, skill taxonomy

2

Deep Job Understanding

Parse each JD → required/preferred skills, experience range, seniority → synthesize ideal profile

3

Contextual Relevance

Shared TF-IDF + LSA space across JDs, ideal profiles & resumes → semantic scores

4

Signal Integration

Skill-match, experience-fit, and pool-normalized behavioral/intent scores per candidate

5

Data Validation

Flag sparse / implausible / duplicate profiles → confidence multiplier + reviewer notes

6

Ranking

Weighted combination → ranked shortlist + plain-English justification per candidate

7

Optional LLM Refinement

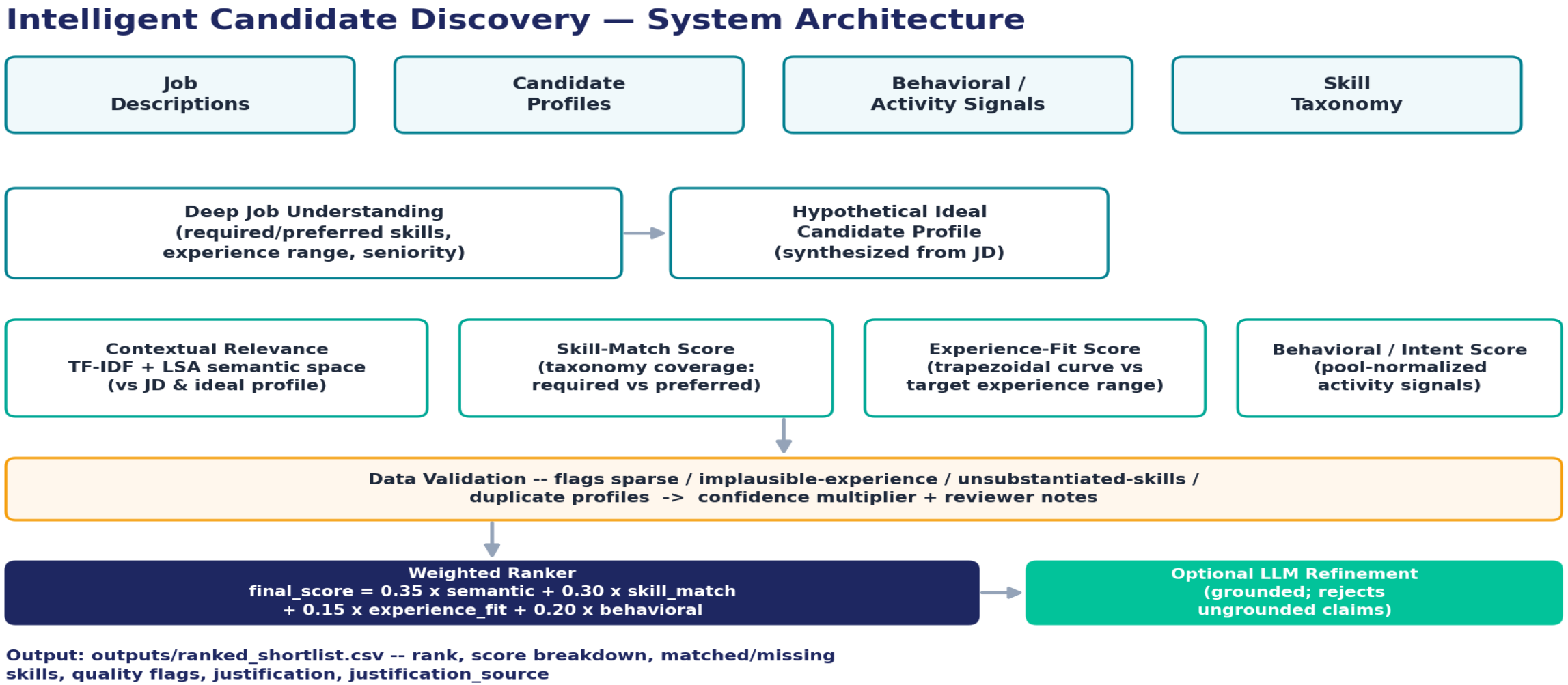
Grounded re-ranking / justification polish for top-N, with automatic hallucination rejection

8

Output

ranked_shortlist.csv — rank, full score breakdown, matched skills, quality flags, justification

System Architecture



Results & Performance

Top 1/3

Hidden-gem candidates (off-title, real ML substance) rank in the top third — ahead of a dozen+ adjacent-domain candidates

< 2 sec

Full pipeline run: ~45 candidates × 3 jobs, fit + score, on a standard CPU — fully offline, no GPU/model downloads

100%

Of ranking decisions are explainable — every score = 4 components + matched/missing skills + plain-English justification

What the included 44-candidate / 3-job demo dataset shows

- A candidate with a 100% required-skill match but a profile dormant for 4+ months ranks below several candidates with lower raw skill overlap but strong recent activity — the behavioral signal is doing real work, not decoration.
- Off-domain candidates (Sales, HR, Content) correctly sink to the bottom of every job's ranking.
- Planted 'suspicious' profiles (sparse, implausible experience, duplicate text) are all correctly flagged and surfaced with explanations.
- Replace with real-dataset numbers once available (see SUBMISSION_GUIDE §4).

Technologies Used

Technology	Why
Python 3	Core implementation language
pandas / numpy	Data loading, feature computation, normalization
scikit-learn (TfidfVectorizer, TruncatedSVD)	Offline semantic embeddings (TF-IDF + LSA) — no model downloads, fully reproducible
PyYAML	Config-driven design — data paths, column mapping, ranking weights all externalized
Streamlit (optional)	Interactive demo dashboard for live evaluation
Anthropic Claude API (optional)	Grounded LLM re-ranking / justification refinement, with automatic hallucination rejection

Why this stack?

Chosen deliberately for an offline-first, dependency-light, fully explainable baseline — architected so production-grade components (sentence embeddings, learning-to-rank) can be swapped in without touching the rest of the pipeline.

Every swap point is documented in the README (§3.2, §3.5).

Submission Assets

GitHub Repository

<your repo URL — make sure it's public>

Demo Video

<link, if recorded — e.g. streamlit run app.py walkthrough>

Ranked Output File

outputs/ranked_shortlist.csv

Thank You

Intelligent Candidate Discovery — India Runs, Track 1

<Team Name> · <contact email / LinkedIn>